Messaging with RabbitMQ

https://www.rabbitmq.com/#getstarted



About RabbitMQ

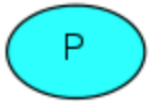
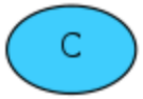
RabbitMQ is an open-source distributed message/streaming broker.

RabbitMQ supports AMQP (Advanced Message Queuing Protocol), MQTT (Message Queue Telemetry Transport protocol)

AMQP, MQTT are an open standard application layer protocols.

RabbitMQ accepts, stores, and forwards *messages*: blobs of data

Jargons

Producing A program which sends message is a *producer.*	
Queue : a post box living inside RabbitMQ.	
Consuming A *consumer* is a program that mostly waits to receive messages.	

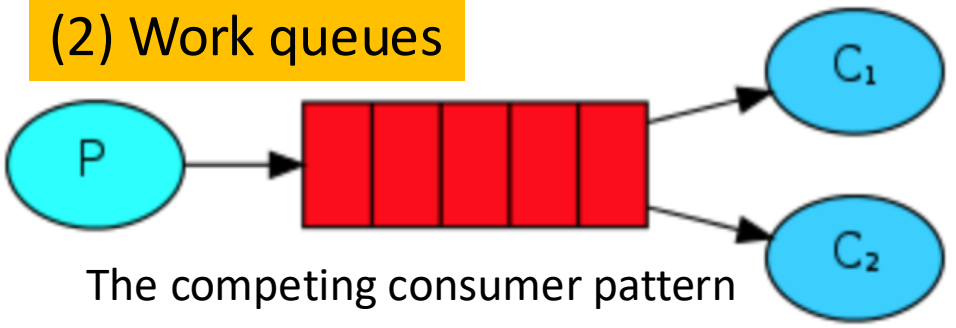
RabbitMQ is a push model.

Queue sends message to consumers

(1) Hello world



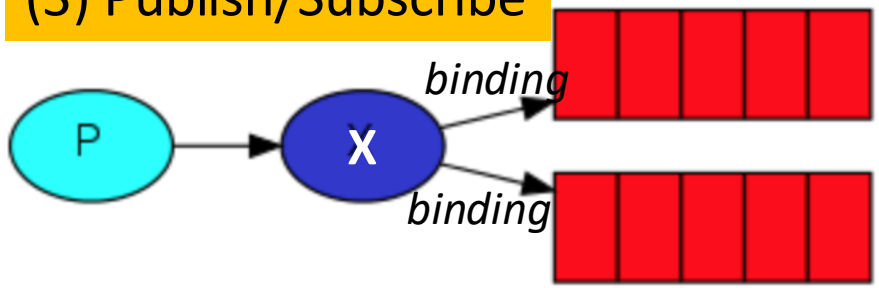
(2) Work queues



The competing consumer pattern

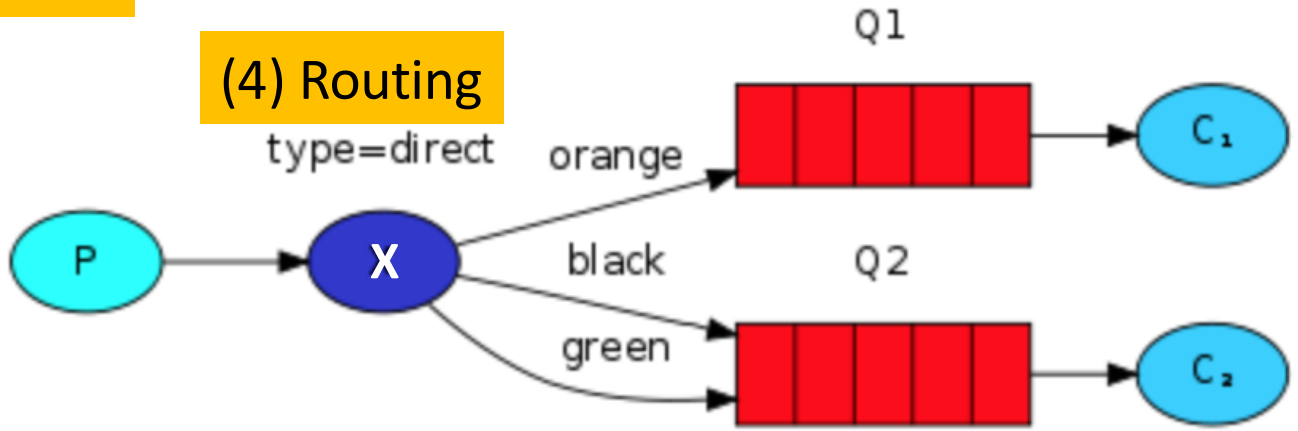
(7) Publisher confirms

(3) Publish/Subscribe



Exchange type: direct, topic, headers, fanout

(4) Routing



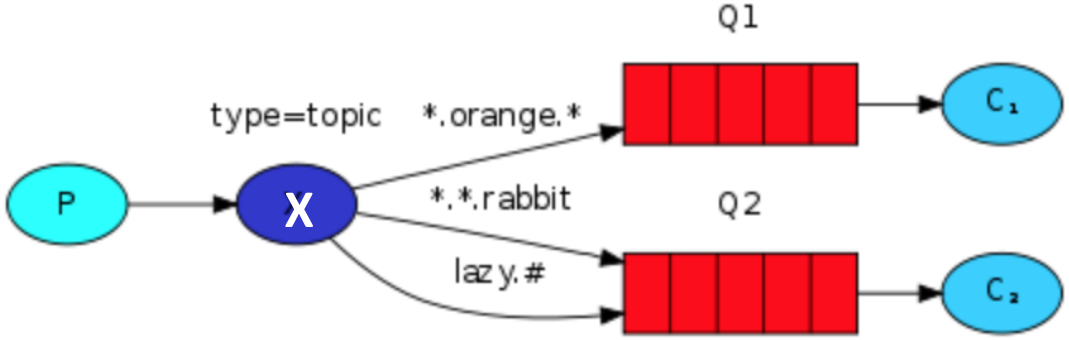
Exchange type: direct, topic, headers, fanout

binding key == routing key

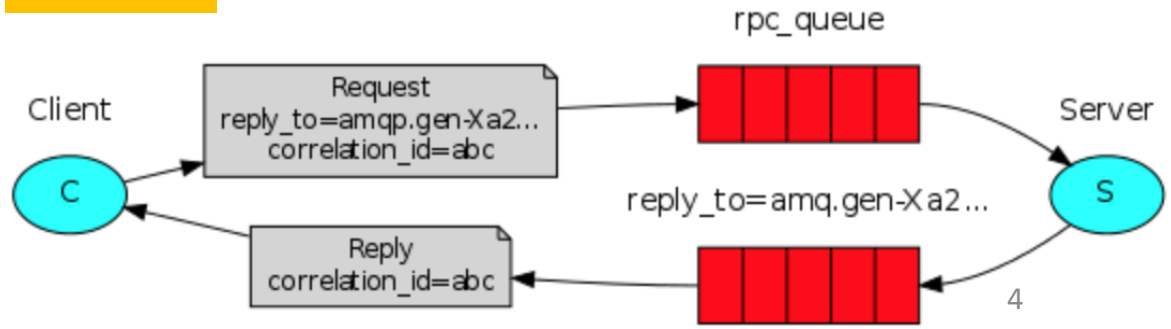
(5) Topics

* Can substitute for exactly one word

\# Can substitute for zero or more words



(6) RPC

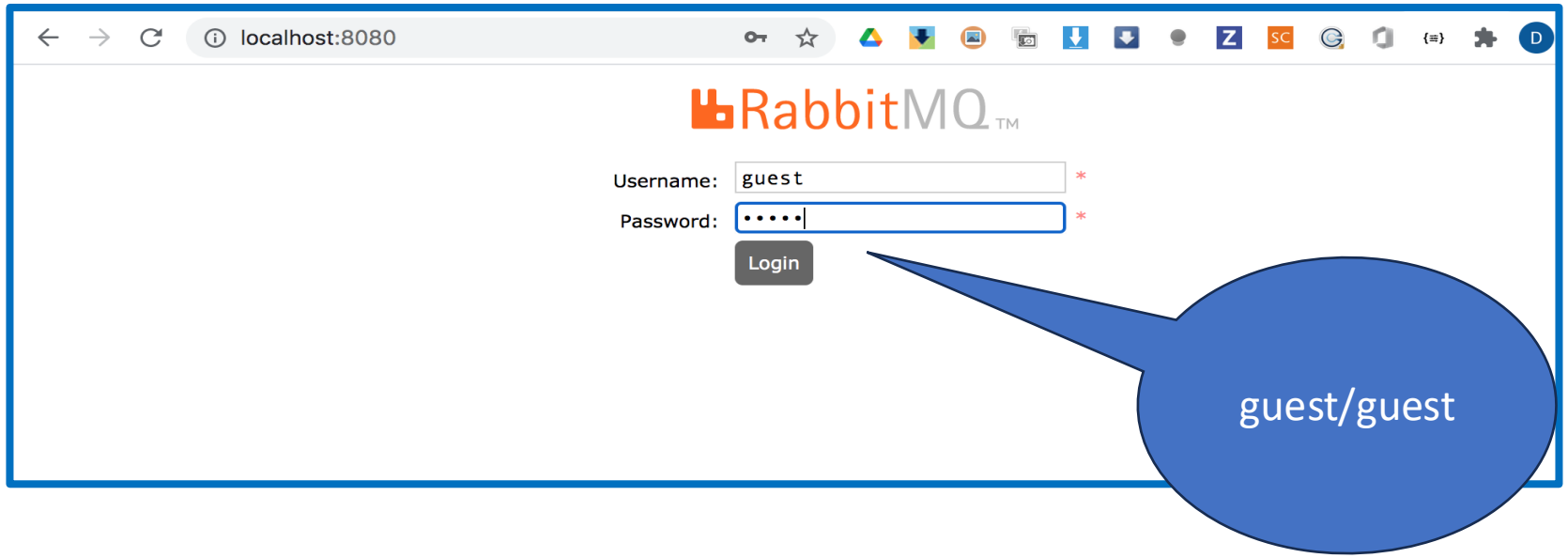


Pre-requisite

```
docker run -d \
  --name rabbitmq-lab \
  --hostname rabbitmq-lab \
  -p 127.0.0.1:5672:5672 \
  -p 127.0.0.1:8080:15672 \
  -v rabbitmq-lab-data:/var/lib/rabbitmq \
  rabbitmq:4.3.6-management
```

https://hub.docker.com/rabbitmq

Pre-requisite

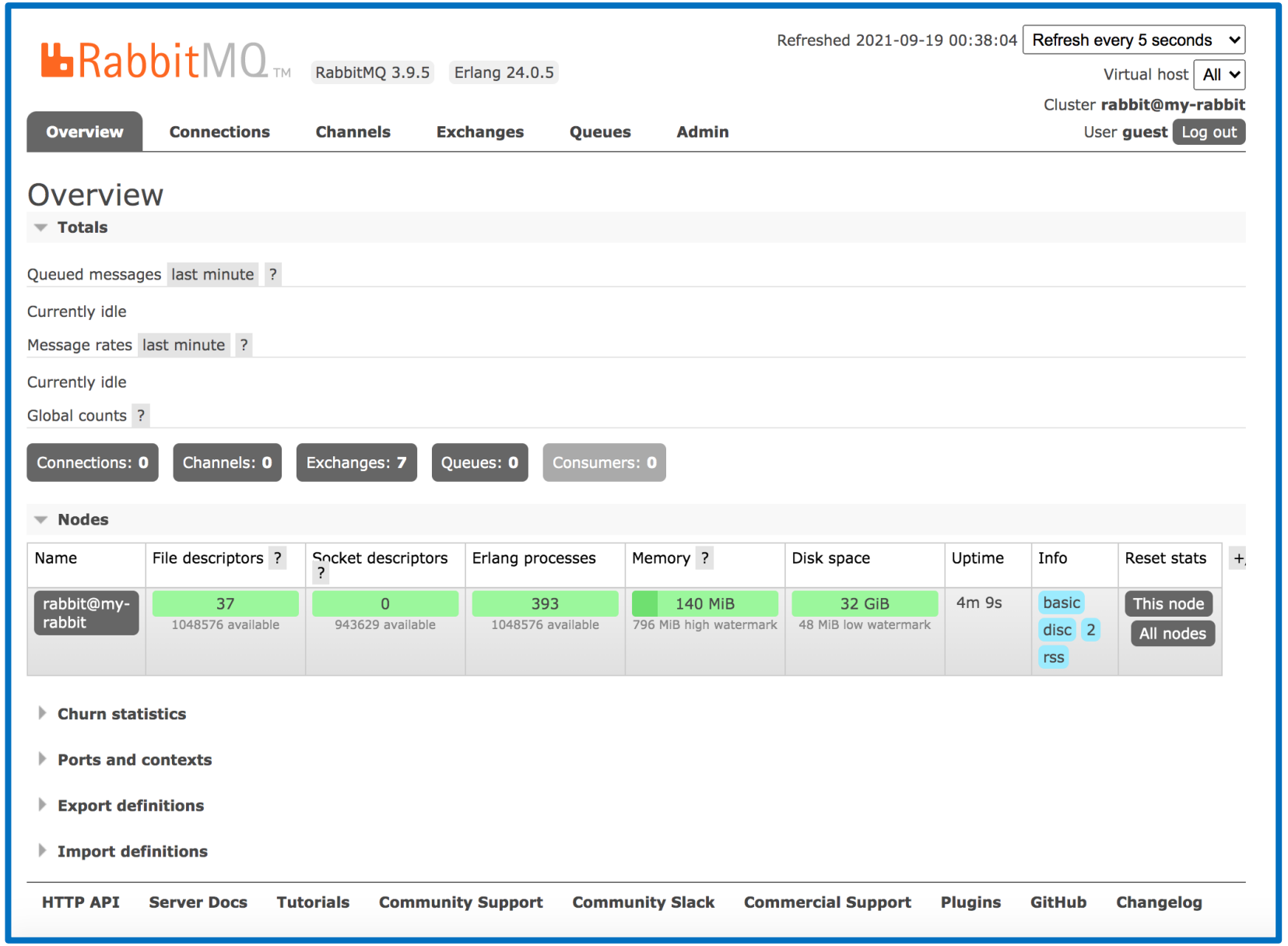


http://localhost:8080

```
docker exec rabbitmq-lab rabbitmqctl status
docker exec rabbitmq-lab rabbitmqctl
```

https://hub.docker.com/rabbitmq

RabbitMQ Management



Python library as RabbitMQ client

```
python -m pip install pika -upgrade
```

pika uses AMQP 0-9-1

(1)
Hello world

Producer (P)

```python
import sys

#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
channel.queue_declare(queue='hello', durable=True)

channel.basic_publish(exchange='',
                      routing_key='hello',
                      body='Hello World!')
print(" [x] Sent 'Hello World!'")
connection.close()
```

(1) Hello world




```
python 1_HelloReceive.py
python 1_HelloSent.py
```

(1)
Hello world

Consumer (C)

```python
#!/usr/bin/env python
import pika, sys, os

def main():
    connection = pika.BlockingConnection(pika.ConnectionParameters(host='localhost'))
    channel = connection.channel()

    channel.queue_declare(queue='hello', durable=True)

    def callback(ch, method, properties, body):
        print(" [x] Received %r" % body)

    channel.basic_consume(queue='hello', on_message_callback=callback, auto_ack=True)

    print(' [*] Waiting for messages. To exit press CTRL+C')
    channel.start_consuming()

if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted')
        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)
```

This lab uses push delivery through basic_consume. RabbitMQ also offers polling through basic_get()

(1) Hello world




```
python 1_HelloReceive.py
python 1_HelloSent.py
```

(1)
Hello world

DEMO

(1) Hello world



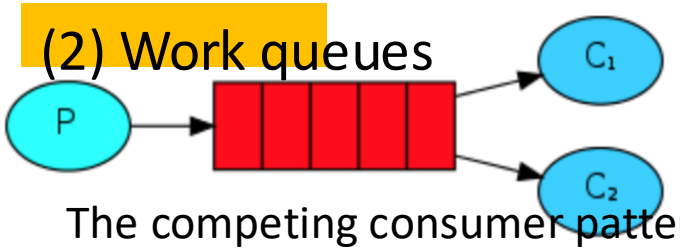

```
python 1_HelloReceive.py
python 1_HelloSent.py
```

(2) Work queue/ Task queue

The assumption behind a work queue is that each task is delivered to exactly one worker.

Round robin dispatching

(2) Work queues

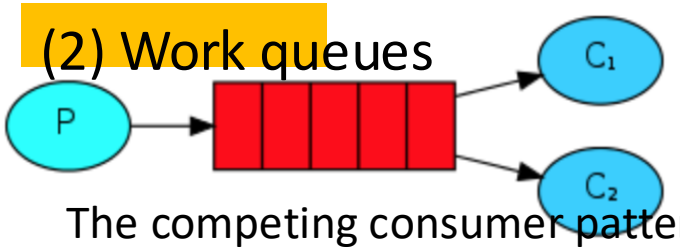


The competing consumer pattern

```
python 2_Worker.py
python 2_Worker.py
…
python 2_TaskQueue.py
```

(2) Work queue/ Task queue (cont.)

(2) Work queues



The competing consumer pattern

```
python 2_Worker.py
python 2_Worker.py
…
python 2_TaskQueue.py
```

Producer (P)

```
1   #!/usr/bin/env python
2   import pika
3   import sys
4
5   connection = pika.BlockingConnection(
6       pika.ConnectionParameters(host='localhost'))
7   channel = connection.channel()
8
9   channel.queue_declare(queue='task_queue', durable=True)
10                                          #make queue persistent
11  message = ' '.join(sys.argv[1:]) or "Hello World!"
12  channel.basic_publish(
13      exchange='',
14      routing_key='task_queue',
15      body=message,
16      properties=pika.BasicProperties(
17          delivery_mode=2,  # make message persistent
18      ))
19  print(" [x] Sent %r" % message)
20  connection.close()
```

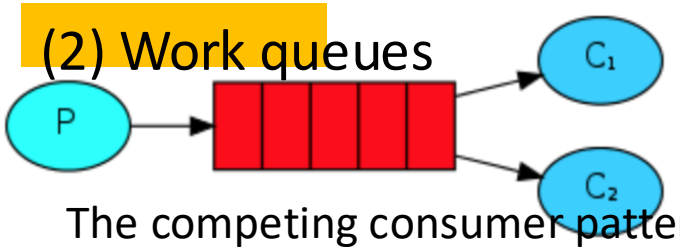
To ensure if RabbitMQ quits or crashes, it won't forget queues

Message will be stored to disk.

pika.spec.PERSISTENT_DELIVERY_MODE

(2) Work queue/ Task queue (cont.)

(2) Work queues



The competing consumer pattern

```
python 2_Worker.py
python 2_Worker.py
…
python 2_TaskQueue.py
```

Consumer (C)

```python
#!/usr/bin/env python
import pika
import time

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.queue_declare(queue='task_queue', durable=True)
print(' [*] Waiting for messages. To exit press CTRL+C')


def callback(ch, method, properties, body):
    print(" [x] Received %r" % body.decode())
    time.sleep(body.count(b'.'))
    print(" [x] Done")
    ch.basic_ack(delivery_tag=method.delivery_tag)

channel.basic_qos(prefetch_count=1)
channel.basic_consume(queue='task_queue', on_message_callback=callback)

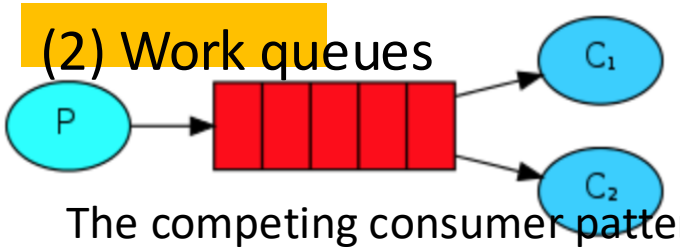
channel.start_consuming()
```

To ensure if RabbitMQ quits or crashes, it won't forget queues

To ensure that worker will send ack when completing the task.

(2) Work queue/ Task queue (cont.)

(2) Work queues



The competing consumer pattern

```
python 2_Worker.py
python 2_Worker.py
…
python 2_TaskQueue.py
```

Consumer (C)

Publisher confirms tell the producer that RabbitMQ has accepted responsibility for a published message. This example code does not have it.

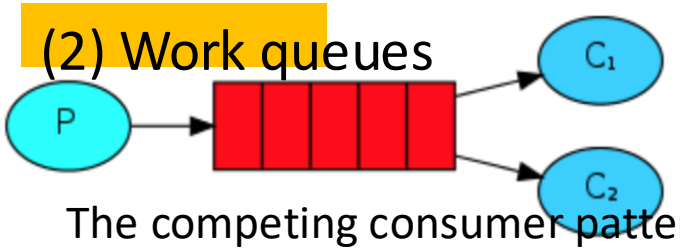
```
1  #!/usr/bin/env python
2  import pika
3  import time
4
5  connection = pika.BlockingConnection(
6      pika.ConnectionParameters(host='localhost'))
7  channel = connection.channel()
8
9  channel.queue_declare(queue='task_queue', durable=True)
10 print(' [*] Waiting for messages. To exit press CTRL+C')
11
12
13 def callback(ch, method, properties, body):
14     print(" [x] Received %r" % body.decode())
15     time.sleep(body.count(b'.'))
16     print(" [x] Done")
17     ch.basic_ack(delivery_tag=method.delivery_tag)
18
19
20 channel.basic_qos(prefetch_count=1)
21 channel.basic_consume(queue='task_queue', on_message_callback=callback)
22
23 channel.start_consuming()
24
```

quits or crashes, it won't forget queues

To ensure that worker will send ack when completing the task.

(2) Work queue/ Task queue (cont.)

(2) Work queues



The competing consumer pattern

```
python 2_Worker.py
python 2_Worker.py
…
python 2_TaskQueue.py
```

Consumer (C)

What the difference between round robin and fair dispatch?

Fair dispatch

```python
#!/usr/bin/env python
import pika
import time

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.queue_declare(queue='task_queue', durable=True)
print(' [*] Waiting for messages. To exit press CTRL+C')


def callback(ch, method, properties, body):
    print(" [x] Received %r" % body.decode())
    time.sleep(body.count(b'.'))
    print(" [x] Done")
    ch.basic_ack(delivery_tag=method.delivery_tag)


channel.basic_qos(prefetch_count=1)
channel.basic_consume(queue='task_queue', on_message_callback=callback)

channel.start_consuming()

```

limits a consumer to one unacknowledged message

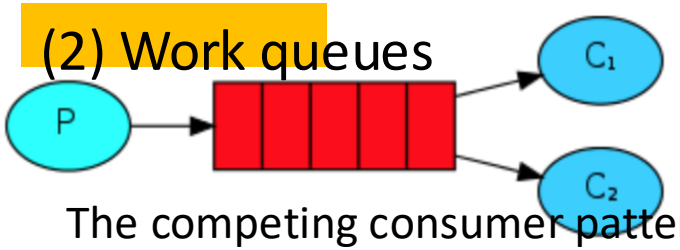
Different stages related to Producer & Message Broekr & Consumer

Mechanism	What it protects	What it does not prove
Durable queue	Queue survives restart	Every message survives
Persistent message	Message is eligible to survive restart in a durable queue	Producer knows it was safely accepted
Publisher confirm	Producer knows the broker accepted responsibility	Consumer completed the work
Consumer acknowledgement	Broker is told processing completed	Business effects happened exactly once

(2) Work queue/ Task queue (cont.)

DEMO

(2) Work queues



The competing consumer pattern

```
python 2_Worker.py
python 2_Worker.py
…
python 2_TaskQueue.py
```

(3) Publish/ Subscribe (fanout)

RabbitMQ will deliver a message to multiple consumers.

```
docker exec rabbitmq-lab rabbitmqctl list_exchanges
docker exec rabbitmq-lab rabbitmqctl list_bindings
```

Producer (P)

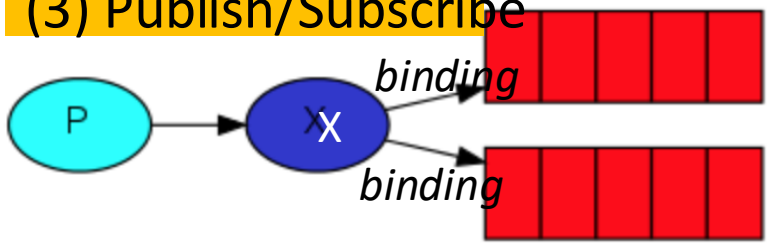
```python
#!/usr/bin/env python
import pika
import sys

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.exchange_declare(exchange='logs', exchange_type='fanout')

message = ' '.join(sys.argv[1:]) or "info: Hello World!"
channel.basic_publish(exchange='logs', routing_key='', body=message)
print(" [x] Sent %r" % message)
connection.close()
```

(3) Publish/Subscribe



Exchange type: direct, topic, headers, fanout

The use of exchange introduces the full messaging model in Rabbit.

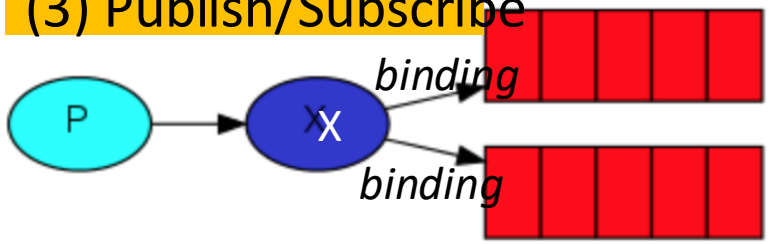
Core idea is that a producer never send any messages directly to a queue..

```
python 3_Publish_broadcast_log.py
python 3_Subscribe_log.py
```

The exchange routes a copy to every bound queue.

(3) Publish/ Subscribe (fanout) (cont.)

(3) Publish/Subscribe



Exchange type: direct, topic, headers, fanout

```
python 3_Publish_ broadcast_log
python 3_Subscribe_log.py
```

Consumer (C)

Temporary subscriptions

We want to hear all messages and only the current flowing messages not the older ones.
แปลว่า?

```python
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.exchange_declare(exchange='logs', exchange_type='fanout')

result = channel.queue_declare(queue='', exclusive=True)
queue_name = result.method.queue

channel.queue_bind(exchange='logs', queue=queue_name)

print(' [*] Waiting for logs. To exit press CTRL+C')

def callback(ch, method, properties, body):
    print(" [x] %r" % body)

channel.basic_consume(
    queue=queue_name, on_message_callback=callback, auto_ack=True)

channel.start_consuming()
```

Let the server choose the random queue name and this queue will be deleted when the consumer's connection is closed.

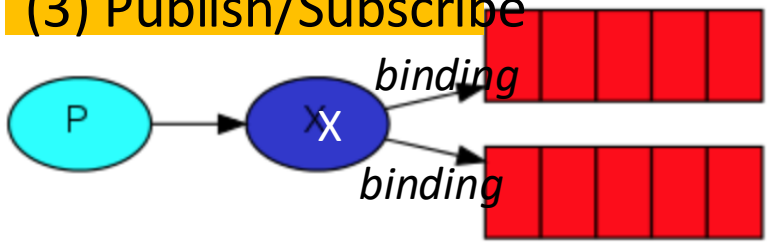
Bind exchange with queue

(3) Publish/ Subscribe (fanout) (cont.)

```
python 3_Subscribe_log.py
python 3_Subscribe_log.py
python 3_Publish_broadcast_log.py
```

DEMO

(3) Publish/Subscribe

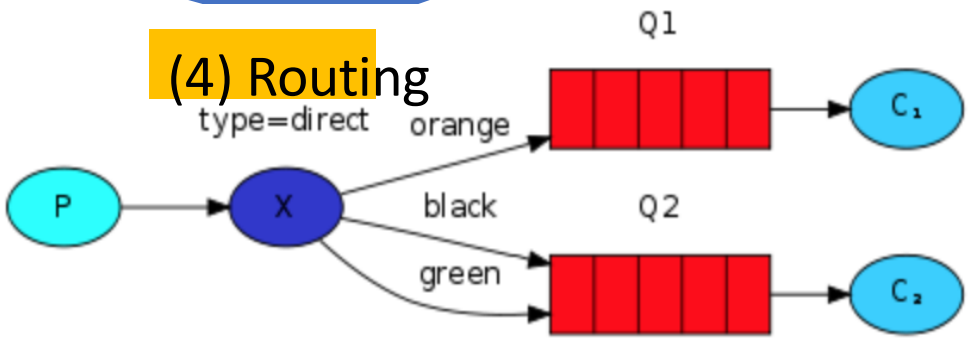


Exchange type: direct, topic, headers, fanout

```
python 3_Publish_broadcast_log.py
python 3_Subscribe_log.py
```

(4) Routing

(4) Routing



Exchange type: direct, topic, headers, fanout
binding key == routing key

Producer (P)

```python
#!/usr/bin/env python
import pika
import sys

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

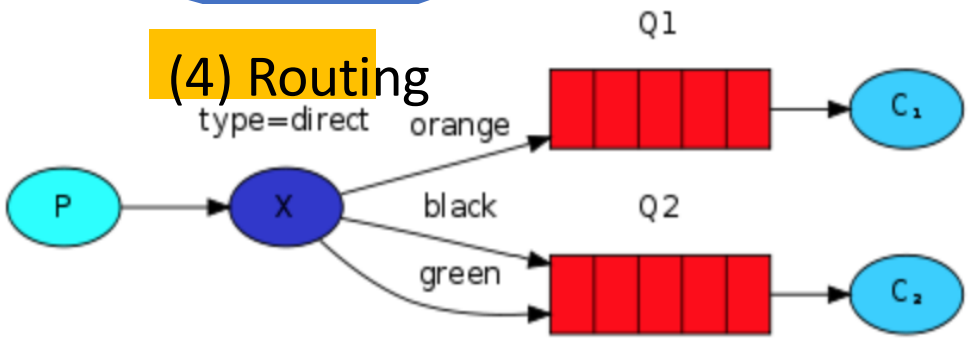
channel.exchange_declare(exchange='direct_logs', exchange_type='direct')

severity = sys.argv[1] if len(sys.argv) > 1 else 'info'
message = ' '.join(sys.argv[2:]) or 'Hello World!'
channel.basic_publish(
    exchange='direct_logs', routing_key=severity, body=message)
print(" [x] Sent %r:%r" % (severity, message))
connection.close()
```

```
python 4_Publish_direct_log.py
python 4_Subscrib_direct_log.py
```

(4) Routing

(4) Routing



Exchange type: direct, topic, headers, fanc
binding key == routing key

```
python 4_Publish_direct_log.py
python 4_Subscrib_direct_log.py
```

Consumer (C)

```python
#!/usr/bin/env python
import pika
import sys

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.exchange_declare(exchange='direct_logs', exchange_type='direct')

result = channel.queue_declare(queue='', exclusive=True)
queue_name = result.method.queue

severities = sys.argv[1:]
if not severities:
    sys.stderr.write("Usage: %s [info] [warning] [error]\n" % sys.argv[0])
    sys.exit(1)

for severity in severities:
    channel.queue_bind(
        exchange='direct_logs', queue=queue_name, routing_key=severity)

print(' [*] Waiting for logs. To exit press CTRL+C')


def callback(ch, method, properties, body):
    print(" [x] %r:%r" % (method.routing_key, body))

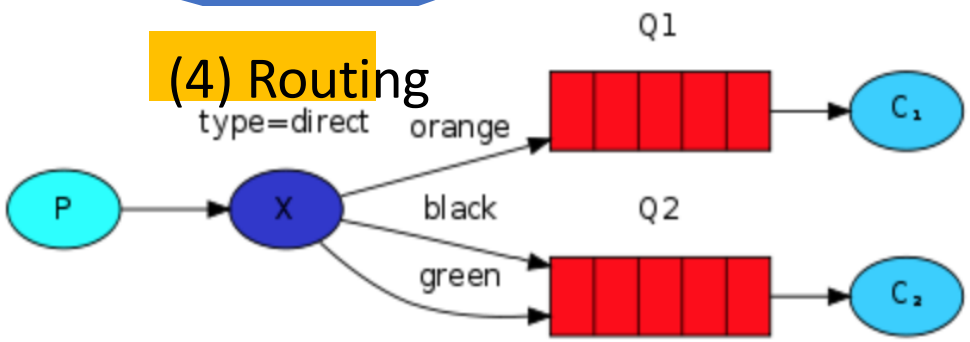

channel.basic_consume(
    queue=queue_name, on_message_callback=callback, auto_ack=True)

channel.start_consuming()
```

(4) Routing

```
python 4_Subscribe_direct_log.py warning error > logs_from_rabbit.log
python 4_Subscribe_direct_log.py info warning error
python 4_Publish_direct_log.py error "GPU card 0 is down!!"
python 4_Publish_direct_log.py info "Add new GPU card!!
```

(4) Routing



Exchange type: direct, topic, headers, fanou

binding key == routing key

```
python 4_Publish_direct_log.py
python 4_Subscrib_direct_log.py
```

DEMO

(5) Topic

```python
#!/usr/bin/env python
import pika
import sys

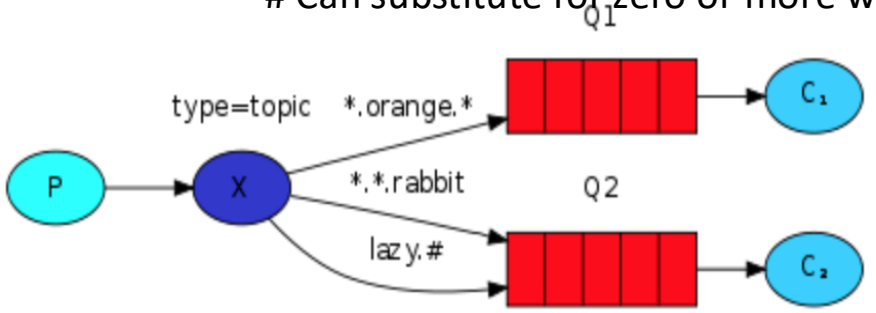
connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.exchange_declare(exchange='topic_logs', exchange_type='topic')

routing_key = sys.argv[1] if len(sys.argv) > 2 else 'anonymous.info'
message = ' '.join(sys.argv[2:]) or 'Hello World!'
channel.basic_publish(
    exchange='topic_logs', routing_key=routing_key, body=message)
print(" [x] Sent %r:%r" % (routing_key, message))
connection.close()
```

(5) Topics

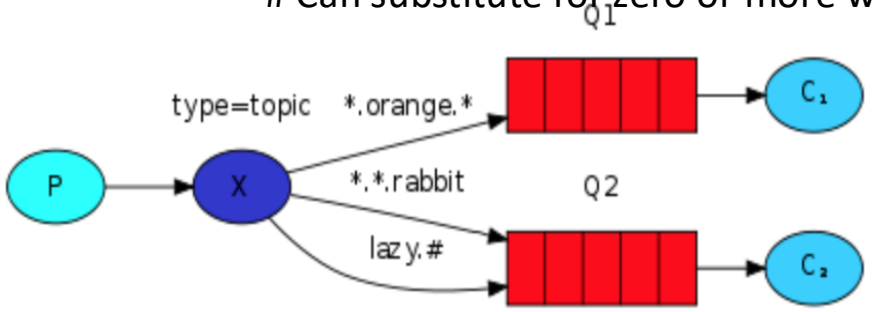
* Can substitute for exactly one word
Can substitute for zero or more words




```
python 5_Publish_topic_log.py
python 5_Subscribe_topic_log.py
```

(5) Topic

(5) Topics * Can substitute for exactly one word
Can substitute for zero or more words




```
python 5_Publish_topic_log.py
python 5_Subscribe_topic_log.py
```

```python
#!/usr/bin/env python
import pika
import sys

connection = pika.BlockingConnection(
    pika.ConnectionParameters(host='localhost'))
channel = connection.channel()

channel.exchange_declare(exchange='topic_logs', exchange_type='topic')

result = channel.queue_declare('', exclusive=True)
queue_name = result.method.queue

binding_keys = sys.argv[1:]
if not binding_keys:
    sys.stderr.write("Usage: %s [binding_key]...\n" % sys.argv[0])
    sys.exit(1)

for binding_key in binding_keys:
    channel.queue_bind(
        exchange='topic_logs', queue=queue_name, routing_key=binding_key)

print(' [*] Waiting for logs. To exit press CTRL+C')


def callback(ch, method, properties, body):
    print(" [x] %r:%r" % (method.routing_key, body))


channel.basic_consume(
    queue=queue_name, on_message_callback=callback, auto_ack=True)

channel.start_consuming()
```

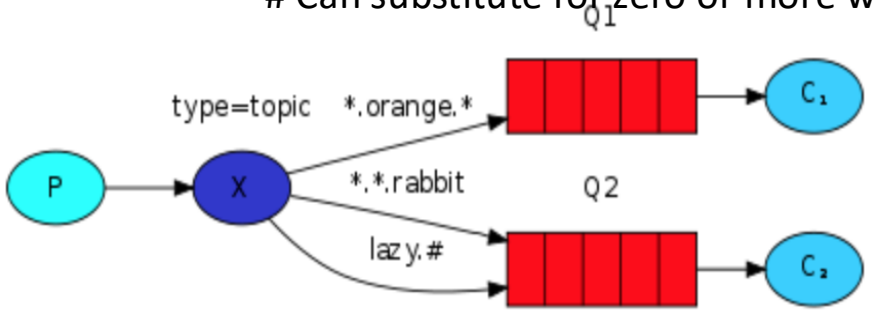
(5) Topic

```
Terminal 1: python 5_Subscribe_topic_log.py "#"
Terminal 2: python 5_Subscribe_topic_log.py "kern.*"
Terminal 3: python 5_Subscribe_topic_log.py "*.critical"
Terminal 4: python 5_Subscribe_topic_log.py "kern.*" "*.critical"
Terminal 5: python 5_Publish_topic_log.py "kern.critical" "kernel error"
```

DEMO

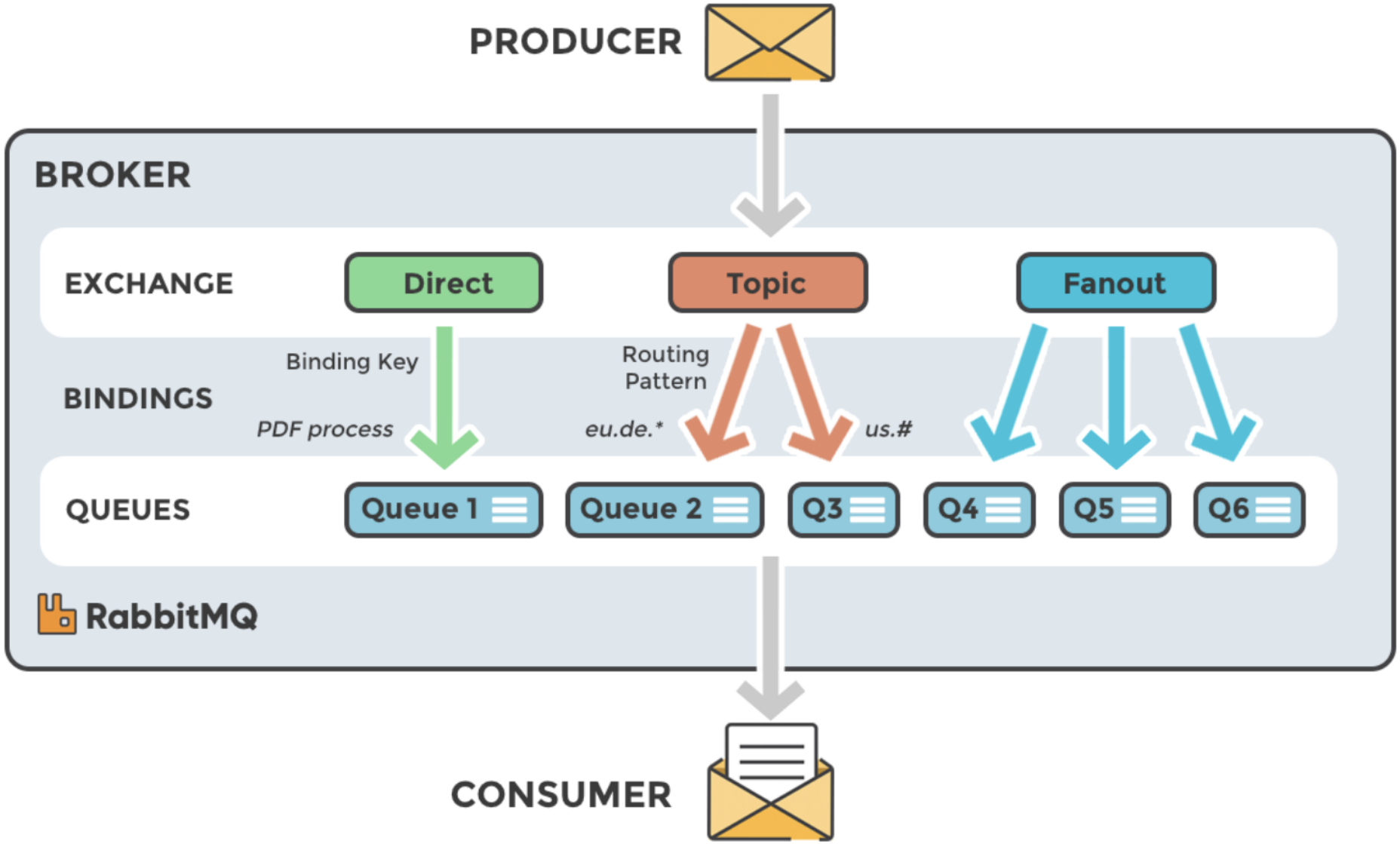
(5) Topics

* Can substitute for exactly one word
Can substitute for zero or more words




```
python 5_Publish_topic_log.py
python 5_Subscribe_topic_log.py
```

RabbitMQ Overall Architecture



Source: https://www.cloudamqp.com/blog/part1-rabbitmq-for-beginners-what-is-rabbitmq.html